4.11 Preventing Accesses via Guessed Addresses

We now show how twin allocation semantics prevents unobserved blocks from being accessed via guessed addresses. This aspect is essential to enable compiler optimizations, otherwise any pointer could potentially access any object.

A block is *unobserved* if any value derived from (i.e., any value that has data or control dependence on) the block's logical address created at allocation is never used in any of the *address-observing* operations: being (*i*) casted to an integer, (*ii*) compared to a physical address, and (*iii*) subtracted to/from a physical address. For example, in the following program, the first block of size `10` is unobserved because its address `a` is never used in an address-observing operation, while the second block of size `5` is observed because its address `b` is compared to the physical address `0x200`.

```
1: char *a = malloc(10);
2: char *b = malloc(5);
3: if (b == (char*)0x200)
4:     *(char*)0x100 = 1;
```

A *guessed* address is necessarily a physical address. We will now show that in the twin allocation model, compilers can safely assume that unobserved blocks cannot be accessed via guessed addresses. Since compilers are allowed to assume anything when the source program has an execution raising UB, it suffices to show that whenever an unobserved block b is accessed via a guessed address p in some execution, there is an alternative execution raising UB. The alternative execution is to take the address of the twin block b' instead of that of b at allocation. After allocating blocks b and b', the twin executions (i.e., the original and the alternative) have exactly the same program state except that:

(*i*) the logical address created at allocation corresponds to a different underlying machine address (i.e., that of b and b' respectively), and
(*ii*) validity of b and b' are swapped.

Condition (*i*) does not make any difference in the twin executions because the machine address is only ever used in address-observing operations. More specifically, our semantics is carefully designed in such a way that all operations other than the address-observing ones are independent of the underlying machine address of any logical address. Thus only condition (*ii*) can make a difference during execution. This can happen only when one of the blocks b and b' is accessed via a physical address. Since only one of b and b' is accessible in each of the twin executions, at least one of the executions will raise UB when block b is accessed via a guessed address.

For instance, in the above example, suppose that at line 1, two blocks of size `10` are allocated at `0x100` and `0x150` with the former activated. Then at line 4 the block is successfully accessed via the physical address `0x100`, while in the twin execution where the block at `0x150` is activated, the access at `0x100` at line 4 raises UB. Therefore, the compiler can conclude that the store at line 4 cannot access object `a`.

Finally, we discuss why allocating two blocks is necessary even though one of them is always made invalid. The reason is because in order to have equivalent twin executions modulo accesses via guessed addresses as described above, the twin executions should have identical memory layouts. For example, instead of allocating two blocks, consider allocating a single block in such a way that there is enough space left for allocating another one of the same size; and raising out-of-memory if such allocation is not possible. In this semantics, one may think that we can still simulate a twin execution as described above using the free space. However, it does not work due to different memory layouts. Specifically, in the above example program, suppose that before executing line 1, the memory only has two segments of free space: `0x100` ∼ `0x109` and `0x200` ∼ `0x209`. Then at line 1, the block of size `10` can be allocated at `0x100` or `0x200` because there is enough space left for